

گزارش تمرین اول

درس ربات های متحرک خودگردان هواپایه

دانشجو: امیررضا کاظم لو

شماره دانشجویی: ۴۰۳۱۴۱۹۰۱۳

استاد درس: دکتر جوانمردی، دکتر خادمیان، دکتر امینی

بهار ۱۴۰۵

۱. مقدمه

به صورت کلی هنگام استفاده از phyphox و به طور کلی تلفن همراه ای که از این نرم افزار استفاده می کنیم دستگاه مختصات به این گونه است که محور x تلفن همراه از سمت دکمه power خارج می شود و محور y آن به سمت جلو (بالای) تلفن همراه هست و با قاعده دست راست و گرفتن انگشت شست به سمت محور x و انگشت اشاره را به سمت محور y بگیریم با جمع کردن دست به محور z می رسیم که همواره از screen تلفن همراه به بیرون می زند؛ بنابراین اگر تلفن همراه را بر روی زمین قرار دهیم انتظار داریم که عددی که در راستای محور z نشان می دهد $+9.8$ باشد.

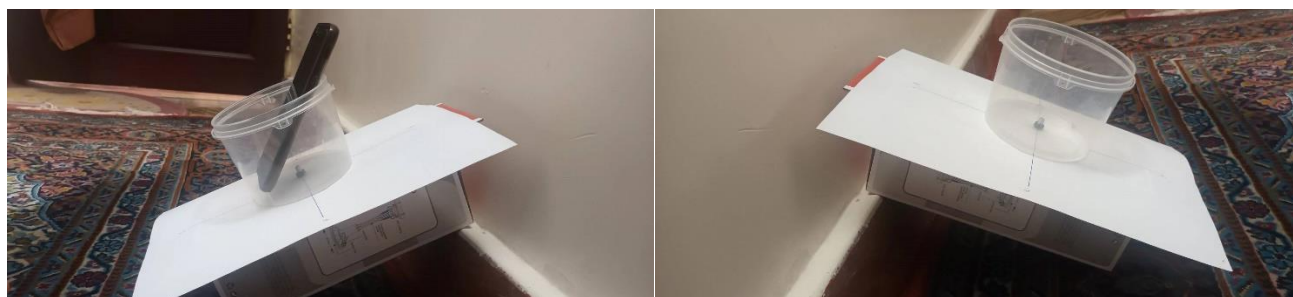
چون ما گوشی را در یک ظرف با سطح شیب دار قرار داده ایم و خود گوشی هم در ظرف زاویه دارد پس در راستای های role, pitch, yaw با چرخاندن گوشی، تغییر خواهیم داشت ولی اگر پایه زیرین یک جای مسطح باشد با چرخش اش صرفاً تغییر در yaw (چرخش حول محور z) خواهیم داشت و role و pitch ثابت خواهد بود؛ پس اگر بخواهیم بدون حرکت انتقالی role, pitch رو هم تغییر دهیم لازم داریم که این سطح شیب دار را داشته باشیم

برای این تمرین لازم است که از سه تا سنسور استفاده کنیم شتاب سنج (accelerator)، مغناطیس سنج (magnetometer) و سرعت زاویه ایی سنج (Gyroscope) که برای داشتن این به صورت ترکیبی می توان یک experiment جدید تعریف کرد و این موارد را فعال کرد تا به صورت synchronized از این سنسور ها داده برداری کنیم.

در این آزمایش به صورت Timed run اجرا می کنیم که قبل از داده برداری تاخیری را ایجاد می کند تا به اون حالتی که مدنظر است برسایم و برای خروجی گرفتن از حالت csv (comma, decimal point) استفاده می کنیم.

۲. پلتفرم پیاده سازی

برای پیاده سازی این آزمایش از پلتفرمی که در شکل زیر هست استفاده شده است:





خروجی این آزمایش در فایل phyphox-۰۴-۰۴-۲۰۲۶ ۰۵-۴۷-۲۵-۰۵ ذخیره شده است که به صورت سه فایل csv است که به صورت synchronized با هم نمونه برداری شده اند.

نکته مهمی که وجود دارد این است که بعد از باز کردن فایل های csv متوجه شدم که تعداد سطر ها با هم دیگر متفاوت است به این معنی که نمونه برداری با نرخ متفاوت در سنسور های مختلف انجام شده است. در همین راستا باید بر اساس زمان (Time (s) کار کنیم، و در جاهایی که لازم است از interpolation یا هم‌تراز سازی زمانی (time alignment) استفاده کنیم.

سوال اول

1- در هر حالت سکون (ایستگاه)، از داده های شتاب‌سنج ها و مغناطیس‌سنج ها برای محاسبه وضعیت استفاده کنید.

برای این که roll, pitch, yaw را محاسبه کنیم باید از فرمول های زیر استفاده کنیم:

• Equations

• Angular rotation dynamics

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & s_{\phi}t_{\theta} & c_{\phi}t_{\theta} \\ 0 & c_{\phi} & -s_{\phi} \\ 0 & s_{\phi}/c_{\theta} & c_{\phi}/c_{\theta} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad \dot{q}(t) = \frac{1}{2}q(t) \otimes \omega(t)$$

• Direct Attitude measurements

$$\begin{aligned} \phi &= \arctan2(a_y, a_z) \\ \theta &= \arctan2(-a_x, \sqrt{a_y^2 + a_z^2}) \end{aligned}$$

$$\begin{aligned} \psi &= \arctan2(-b_y, b_x) \\ &= \arctan2(m_z \sin \phi - m_y \cos \phi, m_x \cos \theta + \sin \theta (m_y \sin \phi + m_z \cos \phi)) \end{aligned}$$

که برای داده های $roll(\phi), pitch(\theta)$ تنها به داده های شتاب سنج نیاز داریم و برای محاسبه $yaw(\psi)$ نیاز به داده های مغناطیس سنج هم داریم.

توضیحات فرمول ها:

۱-ماتریس انتقال:

پارامترها:

- $\dot{\phi}, \dot{\theta}, \dot{\psi}$: اینها نمایانگر سرعت های زاویه ای (نرخ های تغییر) حول محورهای $roll(\phi)$, $pitch(\theta)$ و $yaw(\psi)$ هستند.
- p, q, r : اینها نرخ های چرخش بدنه در اطراف محورهای x, y, z (نرخ های رول، پیچ و یاو) در چارچوب بدنی شیء هستند.
- s_{ϕ}, c_{ϕ} : به ترتیب $\sin(\phi)$ و $\cos(\phi)$ هستند.
- t_{θ} : که $\tan(\theta)$ هست که نسبت $\sin, \cos$ زاویه θ هستند.

$$2- \text{فرمول } \dot{q}(t) = \frac{1}{2}q(t) \otimes \omega(t)$$

پارامتر ها به صورت زیر هستند:

این فرمول نشان دهنده نحوه تغییرات جهت جسم در طول زمان با استفاده از کواترنیون ها و داده های ژيروسکوپ است. پارامترهای آن عبارتند از:

- $\dot{q}(t)$: مشتق زمانی کواترنیون در لحظه t .

این بردار نشان می دهد که وضعیت یا جهت قرارگیری دستگاه با چه نرخ در حال تغییر است.

- $q(t)$: کواترنیون وضعیت دستگاه در لحظه t .

کواترنیون یک بردار ۴ بعدی به فرم $q = [q_w, q_x, q_y, q_z]$ است که به جای زوایای اوایلر (رول، پیچ، یاو) برای نمایش بدون مشکل (مانند قفل گیمبال) دوران ها استفاده می شود.

- $\otimes$: نماد ضرب کواترنیونی (*Quaternion Multiplication*). این ضرب با ضرب ماتریسی یا برداری معمولی تفاوت دارد و قوانین خاص خود را دارد.

- $\omega(t)$: بردار سرعت زاویه‌ای در لحظه t که توسط ژيروسکوپ اندازه‌گیری می‌شود. در این فرمول، $\omega(t)$ به شکل یک کواترنیون خالص (Pure Quaternion) با بخش حقیقی صفر نوشته می‌شود، یعنی: $\omega = [0, \omega_x, \omega_y, \omega_z]$ که در آن $\omega_x, \omega_y, \omega_z$ خروجی‌های سنسور ژيروسکوپ روی سه محور هستند.

۳- برای فرمول‌های

$$Roll(\phi): \phi = \arctan^2(a_y, a_z)$$

$$Pitch(\theta): \theta = \arctan^2(-a_x, \sqrt{a_y^2 + a_z^2})$$

$$Yaw(\psi): \psi = \arctan^2(-b_y, b_x) \\ = \arctan^2(m_z \sin \phi - m_y \cos \phi, m_x \cos \theta + \sin \theta (m_y \sin \phi + m_z \cos \phi))$$

این سه فرمول، فرمول‌های اصلی هستند که برای پیاده‌سازی در ادامه استفاده می‌کنیم.

پارامترها به صورت زیر هستند:

a_x, a_y, a_z : مقادیر شتاب اندازه‌گیری شده توسط سنسور شتاب‌سنج در راستای محورهای X و Y و Z بدنه گوشی. در حالت سکون، این بردار فقط نشان‌دهنده بردار جاذبه زمین است.

m_x, m_y, m_z : مقادیر میدان مغناطیسی اندازه‌گیری شده توسط سنسور مغناطیس‌سنج در راستای محورهای X و Y و Z بدنه گوشی.

b_x, b_y : مولفه‌های افقی میدان مغناطیسی. از آنجا که گوشی ممکن است کج باشد (دارای رول و پیچ باشد)، ابتدا باید بردار مغناطیسی با استفاده از زوایای ϕ و θ به روی یک صفحه کاملاً افقی (موازی با سطح زمین) تصویر شود تا زاویه سمت به درستی محاسبه گردد. فرمول طولانی داخل کمانک، دقیقاً همین عملیات تصویرسازی را انجام می‌دهد.

$\arctan^2(y, x)$: برخلاف $\arctan$ معمولی که فقط زوایای بین $-\frac{\pi}{4}$ تا $\frac{\pi}{4}$ را می‌دهد، این تابع با بررسی علامت هر دو ورودی x و y ، زاویه دقیق را در تمام 360° درجه (بین $-\pi$ تا π) به درستی محاسبه می‌کند.

در هر ایستگاه تو گوشی را در یک وضعیت ثابت نگه داشته‌ای. در این حالت:

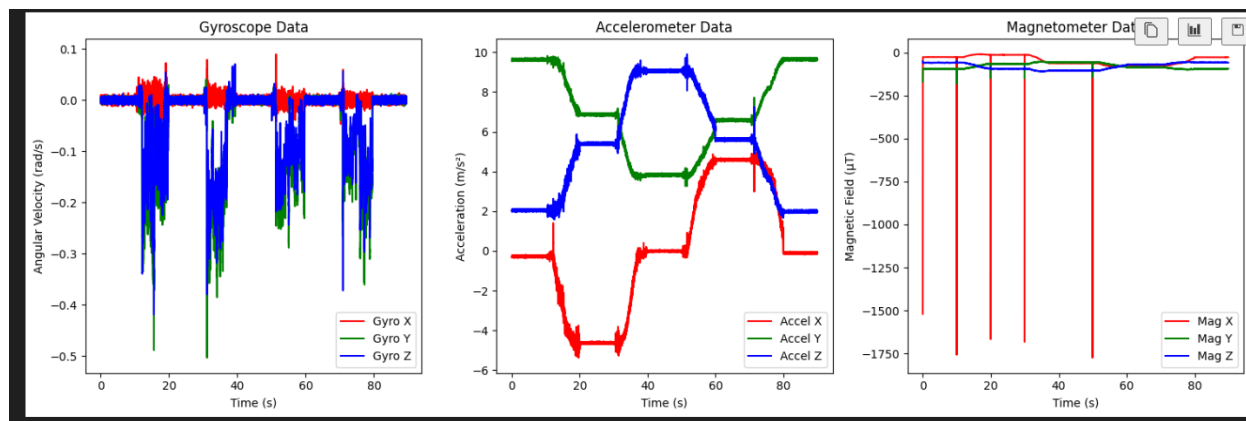
شتاب‌سنج، فقط گرانش را می‌بیند (حرکت کم است $\Rightarrow$ جزء دینامیکی شتاب خیلی کوچک است) یعنی بردار گرانش در دستگاه گوشی:

$$\mathbf{a}_{meas} \approx \mathbf{g}_{body}$$

مغناطیس‌سنج، میدان مغناطیسی زمین را می‌بیند یعنی بردار میدان مغناطیسی در دستگاه گوشی:

$$\mathbf{m}_{meas} \approx \mathbf{B}_{body}$$

کد های پایتون این پروژه در *first_assignment.ipynb* موجود است که در آن ابتدا فایل های *csv* را لود می کنیم و سپس با استفاده از کتابخانه *matplotlib* آن ها را نمایش می دهیم که خروجی آن به صورت زیر می شود:



در نهایت با استفاده از کد های زیر مقادیر *roll*, *pitch*, *yaw* را محاسبه می کنیم

```
Calculate Yaw

def calculate_yaw(mag, roll, pitch):
    mx = mag['mx']
    my = mag['my']
    mz = mag['mz']

    # Adjust for roll and pitch
    m_x = mx * np.cos(pitch) + mz * np.sin(pitch)
    m_y = mx * np.sin(roll) * np.sin(pitch) + my * np.cos(roll) - mz * np.sin(roll) * np.cos(pitch)
    yaw = np.arctan2(-m_y, m_x)

    return yaw

yaw = calculate_yaw(mag_data, roll, pitch)
✓ 0.1s
```

```
Calculating Roll, Pitch, and Yaw

def calculate_roll_pitch(accel):
    ax = accel['ax']
    ay = accel['ay']
    az = accel['az']

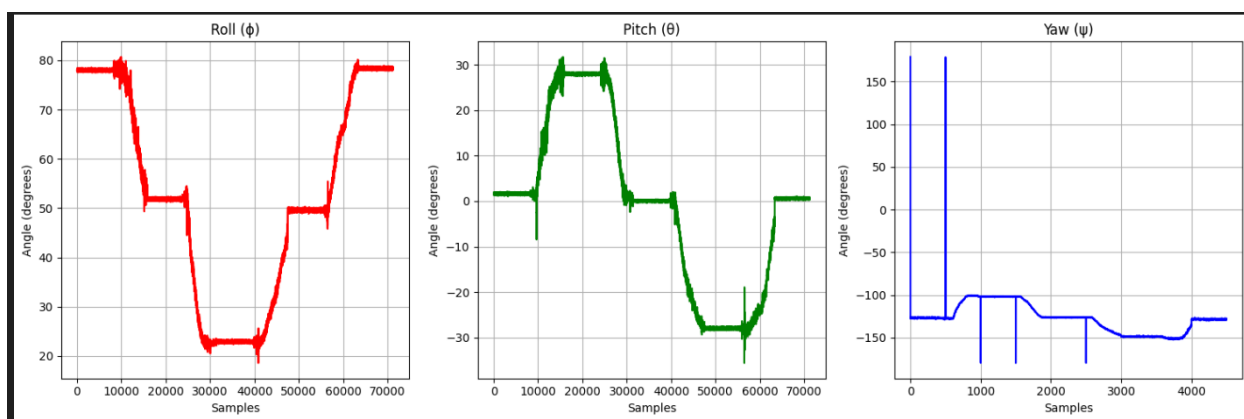
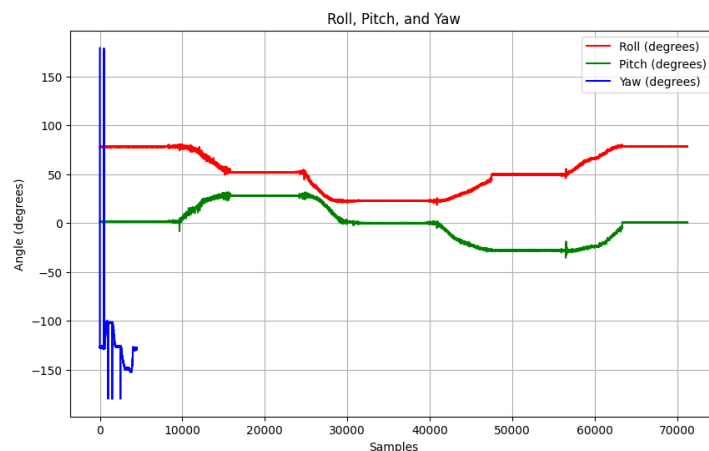
    roll = np.arctan2(ay, az)
    pitch = np.arctan2(-ax, np.sqrt(ay**2 + az**2))

    return roll, pitch

roll, pitch = calculate_roll_pitch(accel_data)
✓ 0.0s
```

که به صورت یک *series* در کتابخانه *pandas* هستند و با استفاده از کتابخانه *matplotlib* آن را نمایش می دهیم (به دو صورت باهم و به صورت جدا از هم) که خروجی آن به صورت زیر می شود (دقت شود که نرخ نمونه برداری

در magnetometer پایین تر بوده است به همین دلیل تعداد sample کمتر در نموداری که همراه با هم رسم شده است بازه کوتاه تری دارد):



نکته مهم این است که این دیتا ها در تمام زمان ها محاسبه شده اند ولی فقط در زمان های ایستا معتبر هستند که در شکل به صورت یک خط افقی هستند و باقی زمان ها خیلی معتبر نیستند بنابراین

در ایستگاه اول $\text{roll}=77,3$, $\text{pitch}=3,8$, $\text{yaw}=-122$

در ایستگاه دوم $\text{roll}=52,8$, $\text{pitch}=29,5$, $\text{yaw}=-100,3$

در ایستگاه سوم $\text{roll}=23,4$, $\text{pitch}=0,9$, $\text{yaw}=-119,8$

در ایستگاه چهارم $\text{roll}=50,1$, $\text{pitch}=-28,8$, $\text{yaw}=-149,4$

در ایستگاه پنجم $\text{roll}=78,3$, $\text{pitch}=0,4$, $\text{yaw}=-122,4$

این ارقام تقریبی و از روی خواندن نمودار بدست آمده است.

روش دوم

در این روش به جای انجام محاسبات بر اساس نمونه ها بر اساس زمان این کار انجام می شود چون نرخ نمونه برداری مغناطیس سنج و شتاب سنج متفاوت است:

خروجی ها در این روش در نقاط ایستا به صورت زیر شد:

in station ۱, the roll degree is ۷۸,۰۰۰۴۷۰۶۳۰۲۸۷۰۱, the pitch degree is ۱,۶۲۱۰۹۰۴۷۴۸۹۰۱۷۸, the yaw degree is -۱۳۱,۲۴۸۰۹۰۶۴۷۰۱۸۸۴

in station ۲, the roll degree is ۵۱,۸۲۸۳۹۲۹۵۷۵۵۵۴۴, the pitch degree is ۲۷,۹۳۱۸۸۹۷۱۲۸۴۳۸۸۳, the yaw degree is -۱۰۲,۷۰۳۵۷۳۱۹۵۹۵۱

in station ۳, the roll degree is ۲۲,۹۰۳۸۵۸۷۴۶۰۷۴۱۸, the pitch degree is ۰,۰۴۹۷۵۷۶۵۸۱۲۶۱۴۲۶۹, the yaw degree is -۱۲۶,۲۲۹۶۶۶۱۱۰۵۰۶۲۲

in station ۴, the roll degree is ۴۹,۵۹۴۰۸۸۷۵۹۳۳۸۹۸, the pitch degree is -۲۷,۹۳۱۰۹۳۸۳۷۱۵۰۰۱, the yaw degree is -۱۴۶,۹۸۹۴۰۵۲۵۸۲۰۸۸۲

in station ۵, the roll degree is ۷۸,۳۶۳۰۵۱۴۸۰۴۳۵۶۹, the pitch degree is ۰,۵۸۲۸۵۶۳۱۰۰۵۱۴۳۴۸, the yaw degree is -۱۲۶,۹۵۴۶۵۷۸۶۶۱۵۷۱

تقریبا همان دیتا هایی هست که به طور تقریبی از روی نمودار خوانده ایم.

تفاوت در این روش با روش قبل این است که دقیقا زمان های مشخص را فقط استفاده کردیم که در حالت ثابت بوده است:

```
t_start_1 = 0.0
t_end_1 = 10.0

t_start_2 = 20.0
t_end_2 = 30.0

t_start_3 = 40.0
t_end_3 = 50.0

t_start_4 = 60.0
t_end_4 = 70.0

t_start_5 = 80.0
t_end_5 = 90.0

acc_st1 = acc[(acc["Time (s)"] >= t_start_1) & (acc["Time (s)"] <= t_end_1)]
mag_st1 = mag[(mag["Time (s)"] >= t_start_1) & (mag["Time (s)"] <= t_end_1)]

acc_st2 = acc[(acc["Time (s)"] >= t_start_2) & (acc["Time (s)"] <= t_end_2)]
mag_st2 = mag[(mag["Time (s)"] >= t_start_2) & (mag["Time (s)"] <= t_end_2)]

acc_st3 = acc[(acc["Time (s)"] >= t_start_3) & (acc["Time (s)"] <= t_end_3)]
mag_st3 = mag[(mag["Time (s)"] >= t_start_3) & (mag["Time (s)"] <= t_end_3)]

acc_st4 = acc[(acc["Time (s)"] >= t_start_4) & (acc["Time (s)"] <= t_end_4)]
mag_st4 = mag[(mag["Time (s)"] >= t_start_4) & (mag["Time (s)"] <= t_end_4)]

acc_st5 = acc[(acc["Time (s)"] >= t_start_5) & (acc["Time (s)"] <= t_end_5)]
mag_st5 = mag[(mag["Time (s)"] >= t_start_5) & (mag["Time (s)"] <= t_end_5)]
```

✓ 0.2s

و میانگین گیری انجام دادیم تا نویز را کاهش دهیم:

میانگین‌گیری روی همان بازه برای کاهش نویز چون گوشی ثابت است، سیگنال‌ها تقریباً ثابت‌اند و نویز اطراف یک مقدار می‌چرخد:

```
import numpy as np

g_b_1 = acc_st1[["Acceleration x (m/s^2)",
                 "Acceleration y (m/s^2)",
                 "Acceleration z (m/s^2)"].mean().to_numpy()
```

همچنین نرمال‌سازی انجام دادیم چون فقط جهت این بردار ها اهمیت داشت و نه اندازه شان:

نرمال‌سازی این بردارها

```
g_b_list, B_b_list = [], []
for B_b_u, g_b_u in list(zip(B_b, g_b)):
    g_b_unit = g_b_u / np.linalg.norm(g_b_u)
    B_b_unit = B_b_u / np.linalg.norm(B_b_u)
    g_b_list.append(g_b_unit)
    B_b_list.append(B_b_unit)
```

41] ✓ 0.1s

و باقی موارد مشابه روش اول بود

۱۱۱. سوال دوم

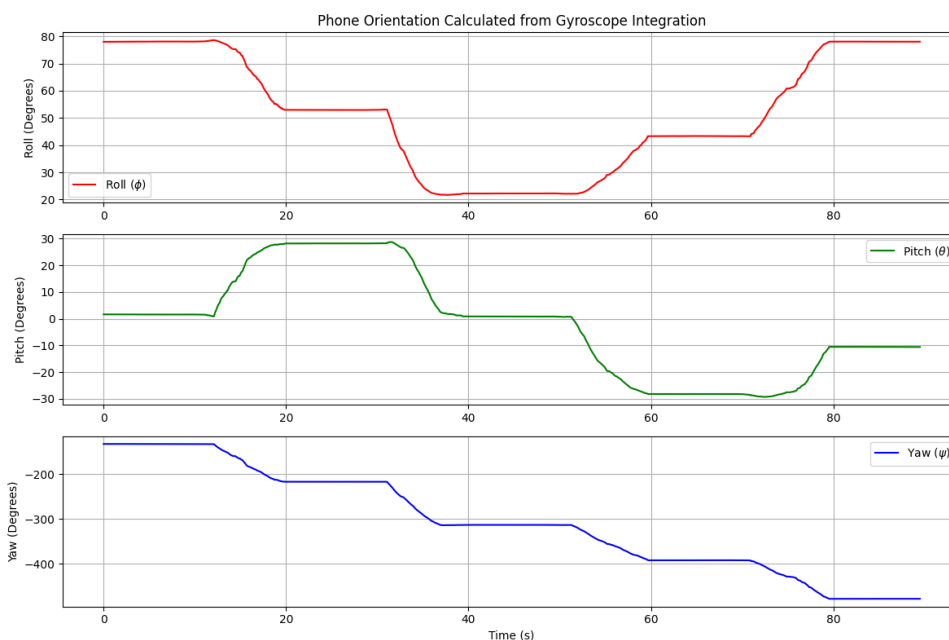
2- با استفاده از خروجی ژایروسکوپ ها در هر لحظه وضعیت گوشی را محاسبه کنید. مقدار وضعیت را برای محاسبه ماتریس دوران اولیه، از خروجی شتاب‌سنج ها و مغناطیس‌سنج ها در حالت سکون ایستگاه اول محاسبه کنید.

برای این منظور باید از فرمول زیر استفاده کنیم که p, q, r خروجی های ژایروسکوپ هستند از طریق معادلات زیر تبدیل بشوند به سرعت در راستای roll, pitch, yaw (ϕ ، θ و ψ) و سپس از آن ها انتگرال گرفته بشوند تا زوایای roll, pitch, yaw بدست بی‌آید. برای محاسبه دوران (در گرفتن انتگرال) نیاز به یک مقدار اولیه داریم که می‌توانیم از ایستگاه سکون اول از طریق خروجی شتاب‌سنج و مغناطیس‌سنج داریم تا انتگرال را بگیریم و مقدار roll, pitch و yaw را به‌روزرسانی کنیم. پس از این طریق می‌توانیم در بازه هایی که متحرک هست در هر لحظه roll, pitch و yaw را محاسبه کنیم.

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & s_{\phi}t_{\theta} & c_{\phi}t_{\theta} \\ 0 & c_{\phi} & -s_{\phi} \\ 0 & s_{\phi}/c_{\theta} & c_{\phi}/c_{\theta} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

در اینجا بیشتر به دنبال پیدا کردن زوایا در زمان هایی هستیم که حرکت انتقالی داریم که دیگر نمی شود از دو فرمول قبلی با استفاده از شتابسنج و مغناطیس سنج به صوت معتبر استفاده کرد.

این کد ها به سه روش اجرا شدن یکی با استفاده از فرمول تبدیل اویلری و دیگری با همین فرمول بالا که در تمام موارد باعث شده است که همان طور که مشخص است خروجی در حالت نهایی با حالت اولیه مقداری فاصله دارد (یعنی state ۵, ۱ است) این است که سنسور ژایروسکوپ (هر چند اندک) دارای خطای و نویز هستند. وقتی برای محاسبه زاویه از داده های ژایروسکوپ انتگرال می گیرید این خطاهای کوچک در طول زمان با هم جمع شده و باعث می شوند زاویه محاسبه شده رفته رفته از مقدار واقعی فاصله بگیرد و Drift کند.



دقت شود که در شکل بالا بعد از ۳۶۰ درجه دوباره صفر می شود پس زاویه ای که yaw نشان می دهد. که در حدود منفی ۱۰۰ خواهد بود مقادیر دقیق این زوایا در ۵ ایستگاه در متن زیر داده شده است.

in timestep ۰, roll is ۷۸,۰۰۰۴۷۰۶۳۰۲۸۷۰۱, pitch is ۱,۶۲۱۰۹۰۴۷۴۸۹۰۱۷۸, yaw is -۱۳۳,۲۹۰۰۷۴۰۴۶۴۱۹۶۲

in timestep ۲۰۰۰۰, roll is ۵۲,۹۳۹۰۹۴۳۹۳۰۶۹۴۳, pitch is ۲۸,۲۳۳۹۰۱۰۹۰۹۹۷۸۷, yaw is -۲۱۷,۴۲۷۰۹۴۵۲۵۷۹۰۳۸

in timestep ۳۰۰۰۰, roll is ۲۱,۷۳۵۶۹۶۶۰۹۵۳۴۴۹۳, pitch is ۱,۷۵۴۱۳۰۷۸۶۵۷۷۱۰۹۹, yaw is -۳۱۳,۹۴۱۴۰۶۹۰۵۴۷۷۸۳

in timestep ۵۰۰۰۰, roll is ۴۳,۳۲۹۰۸۵۹۸۴۳۴۵۶۲۵, pitch is -۲۸,۲۲۹۵۴۶۰۱۵۰۳۴۸۷۴, yaw is -۳۹۲,۰۷۹۶۵۴۳۹۹۲۸۹۰۴

in timestep ۶۰۰۰۰, roll is ۶۱,۳۲۰۳۶۲۱۹۲۲۵۸۱۳, pitch is -۲۷,۳۸۲۲۲۸۹۶۲۵۳۳۵۲, yaw is -۴۲۹,۴۷۴۸۴۱۳۱۰۱۵۷

in timestep ۷۰۰۰۰, roll is ۷۸,۰۵۰۰۱۲۶۸۳۲۴۶۲۸, pitch is -۱۰,۵۹۷۳۵۸۳۸۶۹۵۴۰۳۵, yaw is -
۴۷۷,۷۰۸۵۷۲۵۵۷۳۷۲۷۵

برای قسمت گرفتن انتگرال از روش اویلری برای گرفتن انتگرال به صورت زیر استفاده شده است:

```
# Numerical Integration (Euler Method)
for i in range(1, n_samples):
    dt = time_gyro[i] - time_gyro[i-1]

    # Current angles
    phi = roll_traj[i-1]
    theta = pitch_traj[i-1]
    psi = yaw_traj[i-1] # psi is not needed for the transition matrix but keeping it for completeness

    # To avoid singularity at pitch = +/- 90 degrees (gimbal lock)
    if np.abs(np.cos(theta)) < 1e-6:
        print(f"Warning: Pitch near 90 degrees at index {i}. Potential gimbal lock.")
        theta = np.sign(theta) * (np.pi/2 - 1e-6)

    # Transition Matrix (from the provided image)
    # [ phi_dot ] [ 1 sin(phi)tan(theta) cos(phi)tan(theta) ] [ p ]
    # [ theta_dot ] = [ 0 cos(phi) -sin(phi) ] [ q ]
    # [ psi_dot ] [ 0 sin(phi)/cos(theta) cos(phi)/cos(theta) ] [ r ]

    phi_dot = p[i-1] + np.sin(phi)*np.tan(theta)*q[i-1] + np.cos(phi)*np.tan(theta)*r[i-1]
    theta_dot = np.cos(phi)*q[i-1] - np.sin(phi)*r[i-1]
    psi_dot = (np.sin(phi)/np.cos(theta))*q[i-1] + (np.cos(phi)/np.cos(theta))*r[i-1]

    # Update angles
    roll_traj[i] = roll_traj[i-1] + phi_dot * dt
    pitch_traj[i] = pitch_traj[i-1] + theta_dot * dt
    yaw_traj[i] = yaw_traj[i-1] + psi_dot * dt
```

۱۷. سوال سوم

3- اختلاف زاویه بدست آمده از مراحل 1 و 2 در ایستگاه وضعیت نهایی ناشی از چیست؟ برای رفع آن راه حل ارائه دهید و پیاده سازی نمایید.

تفاوت بنیادی دو روش

انحراف ژيروسکوپ (Gyroscope Drift): ژيروسکوپ‌ها دارای خطای ثابت (Bias) و نویز هستند. وقتی برای محاسبه زاویه از داده‌های ژيروسکوپ انتگرال می‌گیرید ($\theta = \int \omega dt$)، این خطاهای کوچک در طول زمان با هم جمع شده و باعث می‌شوند زاویه محاسبه‌شده رفته‌رفته از مقدار واقعی فاصله بگیرد و Drift کند. به همین دلیل در ایستگاه‌های نهایی خطای ژيروسکوپ بسیار مشهود است.

نویز حرکتی شتاب‌سنج (Accelerometer Noise): شتاب‌سنج‌ها در درازمدت برای یافتن بردار جاذبه زمین بسیار قابل اعتماد هستند، اما در کوتاه‌مدت (وقتی گوشی حرکت می‌کند) علاوه بر جاذبه، شتاب خطی حرکت دست شما را نیز ثبت می‌کنند که باعث ایجاد نویز شدید در محاسبه زاویه می‌شود.

تداخل مغناطیسی: مغناطیس‌سنج نیز تحت تاثیر میدان‌های مغناطیسی محیط اطراف (آهن، وسایل الکترونیکی) قرار می‌گیرد.

همچنین داده سنسور ها دارای نویز هستند و همچنین دارای بایاس که این بایاس با زمان تغییر می کند.